TP 6 – Mini-annuaire d'étudiants

TP global 2/4 – navigation multi-pages

Pr. Ahmed Bentajer

Objectifs

- Naviguer entre plusieurs vues avec Vue Router (route dynamique, 404)
- Combiner recherche et tri dans une seule propriété calculée
- Modifier une donnée partagée depuis une vue « détail »

Cahier des charges

Un annuaire d'étudiants avec deux pages :

- **Liste** (`/`) : formulaire d'ajout, recherche par nom/prénom, tri par nom ou par filière, une carte par étudiant menant vers sa fiche ;
- **Fiche** (`/etudiant/:id`) : détails de l'étudiant, modifiables directement, avec retour à la liste.

Une route « catch-all » affiche une page 404. `StudentCard.vue` est déjà fourni et complet – ce TP porte sur le routeur et les vues, pas sur la décomposition en composants (déjà travaillée au TP3).

`npm install && npm run dev` pour démarrer.

1 Routeur

Complétez `src/router/index.js` en suivant le même schéma qu'au TP4 : trois routes (`/`, `/etudiant/:id`, catch-all), enregistrées dans un routeur créé avec `createWebHistory()`.

2 Liste : recherche, tri, ajout

Dans `StudentListView.vue`, complétez d'une part le formulaire et `ajouterEtudiant()` (mêmes réflexes qu'au TP2/TP5 : `v-model`, `@submit.prevent`, validation avant ajout), et d'autre part la barre d'outils (recherche + `select` de tri liés à `recherche`/`tri`) et la propriété calculée `etudiantsAffiches`, qui doit combiner les deux : filtrer *puis* trier.

3 Fiche détail

Dans `StudentDetailView.vue`, complétez la propriété calculée `student`.

Piège classique

Même piège qu'au TP4 : `$route.params.id` est une chaîne, les `id` du store sont des nombres. Convertissez avec `Number(...)` avant de comparer.

4 Modifier depuis la fiche

Remplacez les trois `<p>` (filière, année, email) par des champs éditables liés avec `v-model` directement sur `student.filiere`, `student.year`, `student.email`.

Pour info

Ça fonctionne, et c'est important de comprendre pourquoi : `student` est une `computed`, mais elle ne *construit* pas une nouvelle valeur – elle renvoie une référence vers un objet qui vit déjà dans `store.students` (créé avec `reactive()`). Modifier un champ de cette référence modifie donc directement la donnée source, et la liste (sur l'autre vue) reflétera le changement au prochain passage dessus. C'est différent du TP1, où `booksTriesParAnnee` construisait un *nouveau* tableau : modifier ce nouveau tableau n'aurait rien changé aux données d'origine. La règle reste la même dans les deux cas – ne mutez jamais une donnée qu'une `computed` a *construite* – elle ne s'applique simplement pas ici, puisqu'on modifie la source, pas une construction dérivée.

TP global suivant : un explorateur d'API REST – premier contact avec fetch et la gestion des erreurs réseau.